

FLOW TRANSAKSI — CUSTOMER JOURNEY

Dokumen Tambahan Aktivasi Payment Gateway Midtrans

WebWeave — AI Web Automation Code Generator | Staging: <https://web-weave-lake.vercel.app> | Juni 2026

1. Ringkasan Alur Transaksi

WebWeave adalah layanan SaaS (Software as a Service) berbasis subscription. Customer melakukan **pembelian paket langganan digital** melalui website. Tidak ada barang fisik, keranjang belanja tradisional, atau pengiriman logistik. Konsep "pilih produk → checkout → bayar" dipetakan ke alur SaaS berikut:



URL Website	<code>https://web-weave-lake.vercel.app</code>
Jenis Transaksi	Subscription digital (paket bulanan/tahunan)
Metode Integrasi	Midtrans Invoice API (server-side) + Payment Link
Endpoint Checkout	<code>POST /api/billing/checkout</code>
Endpoint Webhook	<code>POST /api/billing/midtrans/webhook</code>
Endpoint Status	<code>GET /api/billing/midtrans/status?order_id=...</code>

2. Detail Customer Journey — Step by Step

1

Customer Membuka Website WebWeave

Customer membuka browser dan mengakses `https://web-weave-lake.vercel.app` . Halaman utama (hero mode) ditampilkan dengan tagline "AI Web Automation Code Generator" dan form input objective.

URL: / | Next.js 14 App Router | Vercel hosting (HTTPS)

2

Customer Login / Registrasi

Customer klik avatar di sidebar → login form muncul. Customer bisa login dengan Google OAuth atau email/password via Supabase Auth. Akun baru otomatis mendapat paket Free (5 generasi/bulan).

Auth: Supabase (Google OAuth + Email/Password) | Session disimpan di browser

3

Customer Membuka Halaman Pricing

Customer klik tombol "Pricing" di profile menu, atau tombol "Upgrade" di sidebar. Modal Pricing muncul menampilkan 2 paket berbayar: **Starter** (Rp49.000/bln) dan **Pro** (Rp129.000/bln). Toggle monthly/annual tersedia.

Ul: PricingPage modal component | Data paket dari `src/lib/billing/plans.js`

4

Customer Memilih Paket & Siklus Billing

Customer memilih paket (Starter atau Pro) dan siklus billing (monthly/annual). Customer klik tombol **"Pay Now"** pada paket pilihan. Sistem memvalidasi: (a) customer terautentikasi, (b) paket dipilih lebih tinggi dari paket saat ini — anti double billing, (c) origin request valid — anti CSRF.

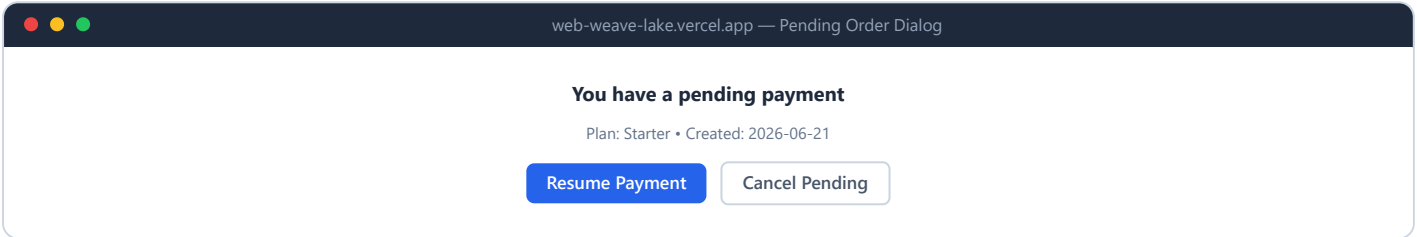
Validasi: `src/lib/server/security.js` (origin check) | `src/lib/billing/quota.js` (plan comparison)

5

Cek Pending Order (Anti Double-Charge)

Sebelum checkout, frontend memanggil `GET /api/account/pending-order` . Jika ada order `checkout_created` sebelumnya: (a) jika status Midtrans sudah terminal (expire/cancel/deny) → server auto-cancel order, lanjut checkout; (b) jika masih aktif → tampilkan dialog "Resume Payment" atau "Cancel". Customer tidak bisa checkout baru sampai pending order diselesaikan.

API: `GET /api/account/pending-order` → cek `billing_orders` `status` `checkout_created` | Midtrans status check: `GET /v2/{order_id}/status`



6

Server WebWeave Membuat Invoice Midtrans

Frontend mengirim `POST /api/billing/checkout` dengan body `{ plan, billingCycle }` (email customer diambil dari session Supabase, bukan input manual). Server generate `order_id` unik (format: `ww_{12charUserIdPrefix}_{10charSha256Hash}`), lalu memanggil **Midtrans Invoice API** `POST /v1/invoices` dengan: `item_details` (nama paket, harga), `customer_details` (email dari session), `payment_type = payment_link`. Midtrans merespons dengan `payment_link_url`. Order disimpan di Supabase `billing_orders` dengan status `checkout_created`.

API: `POST /api/billing/checkout` → `src/lib/billing/midtrans.js` ([createInvoice](#)) | Email: `auth.user.email` dari Supabase session

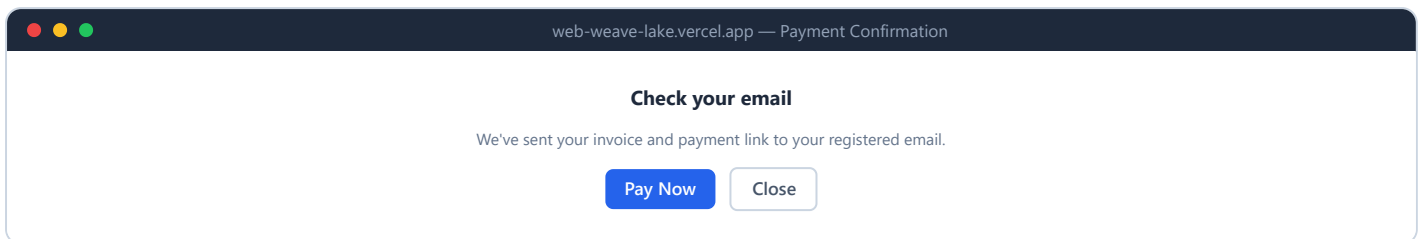


7

Email Konfirmasi & Tombol "Pay Now"

Setelah invoice dibuat, popup konfirmasi muncul menampilkan pesan "Check your email" dan tombol **"Pay Now"**. Popup ini BUKAN form input email — email sudah diambil dari session. Customer klik "Pay Now" untuk langsung redirect ke halaman pembayaran Midtrans, atau cek email untuk link invoice.

UI: `showEmailPopup` ([post-checkout ConfirmDialog](#)) | Redirect: `window.location.href = checkedOutPaymentUrl`

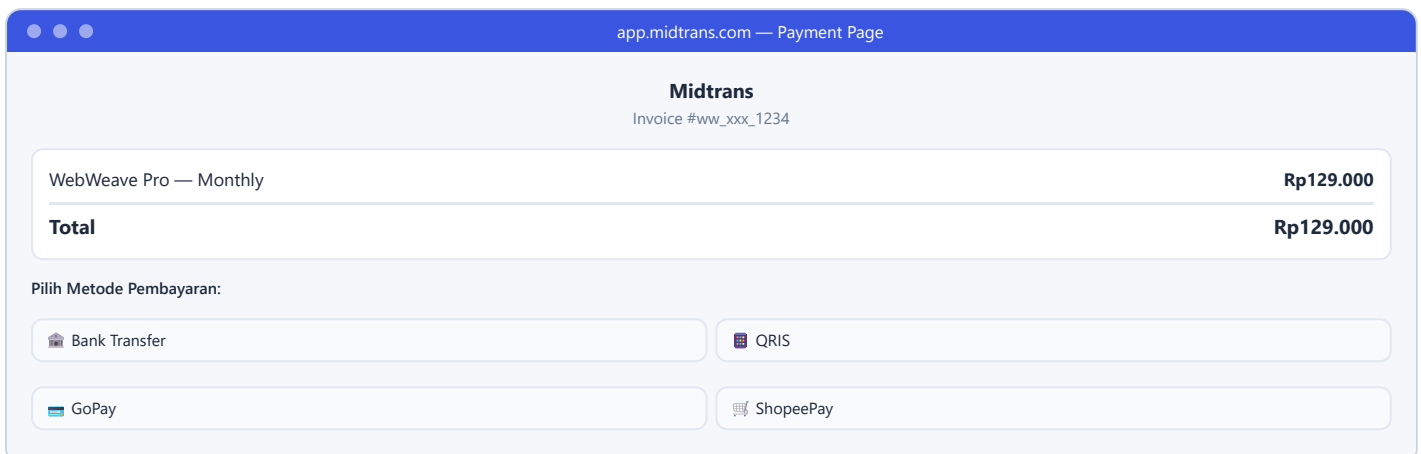


8

Customer Diarahkan ke Halaman Pembayaran Midtrans

Customer klik "Pay Now" → browser redirect ke `payment_link_url` Midtrans. Halaman pembayaran Midtrans menampilkan: detail invoice (paket, harga), pilihan metode pembayaran (transfer bank, QRIS, GoPay, ShopeePay, Akulaku, dll). **Semua data pembayaran diproses oleh Midtrans — WebWeave tidak menyentuh data kartu/apapun.**

Redirect: `window.location.href = payment_link_url` | Domain Midtrans: `app.sandbox.midtrans.com` ([sandbox](#)) / `app.midtrans.com` ([production](#))



9

Customer Menyelesaikan Pembayaran di Midtrans

Customer memilih metode pembayaran dan menyelesaikan transaksi sesuai instruksi Midtrans (transfer ke VA, scan QR, konfirmasi e-wallet, dll). Midtrans memproses pembayaran dan update status transaksi.

Pemroses: [Midtrans \(full\)](#) | WebWeave tidak terlibat dalam tahap ini



10

Midtrans Mengirim Webhook Notifikasi ke WebWeave

Midtrans mengirim HTTP POST notification ke endpoint webhook WebWeave. Server memverifikasi `signature_key` dengan SHA-512 hash (gabungan `order_id` + `status_code` + `gross_amount` + `ServerKey`). Jika valid dan status = `settlement` atau `capture`, akun customer otomatis di-upgrade.

Endpoint: POST `/api/billing/midtrans/webhook` | Verifikasi: SHA-512 signature | Update: Supabase `profiles` table (`plan_id`, `billing_expiry`)



11

Paket Aktif — Customer Menggunakan Layanan

Akun customer ter-upgrade. Kuota generasi bulanan, slot project, dan slot script langsung aktif. Customer kembali ke WebWeave dan dapat langsung menggunakan layanan: input URL target + objective → AI generate automation script. Saat tab browser kembali fokus, sistem auto-refresh status billing.

Auto-refresh: `visibilitychange` event → `loadPrivateData()` | Kuota: `usage_events` table (monthly tracking)

3. Sequence Diagram — API Calls

Berikut adalah urutan panggilan API dari sisi teknis (backend-to-backend):

#	Actor	Action	Detail
1	Customer	Klik "Pay Now"	Frontend cek pending order via <code>GET /api/account/pending-order</code> . Jika ada pending aktif → dialog resume/cancel. Jika terminal → auto-cancel. Lanjut checkout.
2	Frontend → Server	<code>POST /api/billing/checkout</code>	Body: { <code>plan</code> , <code>billingCycle</code> } (email diambil dari session Supabase, bukan input manual)
3	WebWeave Server	Validasi request	Cek auth token (Supabase), cek origin (anti-CSRF via <code>validateOrigin</code>), cek paket saat ini vs pilihan (anti downgrade/double billing via <code>PLAN_TIER</code>)
4	WebWeave Server	Generate <code>order_id</code>	Format: <code>ww_{12charUserIdPrefix}_{10charSha256Hash}</code> — hash dari <code>planId</code> + <code>cycle</code> + <code>timestamp</code> + random bytes. Contoh: <code>ww_a1b2c3d4e5f6_f8k2lm9p0q</code>
4	WebWeave → Midtrans	<code>POST /v1/invoices</code>	Body: <code>item_details</code> (paket, harga), <code>customer_details</code> (email dari session Supabase <code>auth.user.email</code>), <code>order_id</code> , <code>payment_type</code> = "payment_link"
5	Midtrans	Response invoice	Return <code>payment_link_url</code> , <code>invoice_url</code> , <code>pdf_url</code>
6	WebWeave Server	Simpan order	Insert ke Supabase <code>billing_orders</code> : <code>order_id</code> , <code>user_id</code> , <code>plan_id</code> , <code>amount</code> , <code>status</code> = "checkout_created"
7	WebWeave → Customer	Redirect	Return <code>payment_link_url</code> ke frontend → browser redirect ke Midtrans
8	Customer	Bayar di Midtrans	Pilih metode (VA/QRIS/e-wallet), selesaikan pembayaran. Midtrans proses.
9	Midtrans → WebWeave	<code>POST /api/billing/midtrans/webhook</code>	Notification JSON: <code>order_id</code> , <code>transaction_status</code> , <code>signature_key</code> , <code>gross_amount</code>
10	WebWeave Server	Verifikasi signature	<code>SHA-512(order_id + status_code + gross_amount + ServerKey) == signature_key?</code>
11	WebWeave Server	Update akun + cancel pending lain	Jika <code>settlement/capture</code> → update <code>profiles.plan_id</code> , <code>profiles.billing_expiry</code> , reset <code>usage_events</code> kuota. Cancel order <code>checkout_created</code> lain milik user yang sama (anti double-charge)
12	WebWeave Server → Midtrans	ACK webhook	Return HTTP 200 ke Midtrans (konfirmasi webhook diterima)
13	Customer	Kembali ke WebWeave	Tab fokus → auto-refresh → plan badge update → layanan aktif

4. Pemetaan Konsep E-Commerce ke SaaS WebWeave

Midtrans meminta alur "pencarian produk → keranjang → checkout → pembayaran". Berikut pemetaannya ke model SaaS WebWeave:

Konsep E-Commerce (Midtrans)	Padanan di WebWeave (SaaS)	Penjelasan
Customer membuka website/aplikasi	Customer buka <code>web-weave-lake.vercel.app</code>	Akses halaman utama WebWeave via browser
Pencarian & pemilihan produk	Buka halaman Pricing, pilih paket (Starter/Pro)	"Produk" = paket subscription digital. "Pencarian" = browse pricing modal.
Menambahkan produk ke keranjang	Klik "Pay Now" pada paket pilihan	SaaS tidak punya keranjang. 1 paket = 1 transaksi. Klik "Pay Now" = commit to purchase.
Proses checkout	Isi email → server buat Invoice Midtrans	Server-side checkout via Invoice API. Email = customer detail untuk invoice.
Diarahkan ke halaman pembayaran	Redirect ke <code>payment_link_url</code> Midtrans	Customer lanjut bayar di halaman Midtrans (pilih metode, eksekusi)

5. Daftar Paket yang Dijual

Paket	Harga/Bulan	Harga/Tahun	Kuota Generasi	Project Slot	Script Slot
Free	Rp0	Rp0	5/bln	1	0
Starter	Rp49.000	Rp470.000	75/bln	5	5
Pro	Rp129.000	Rp1.238.000	300/bln	25	12

Framework yang didukung: Playwright JS, Playwright Python, Puppeteer JS, Selenium Python, Cypress JS. Paket Free terbatas ke Playwright JS saja. Paket Starter: Playwright JS/Python, Selenium Python, Cypress JS (tidak termasuk Puppeteer JS). Paket Pro: semua 5 framework.

6. Catatan Teknis untuk Midtrans

- **Tipe integrasi:** Invoice API (bukan Snap popup, bukan Core API direct charge).
- **Environment:** Saat ini menggunakan Midtrans Sandbox untuk testing. Production key akan digunakan setelah aktivasi.
- **Webhook URL (production):** `https://web-weave-lake.vercel.app/api/billing/midtrans/webhook`
- **Webhook retry:** WebWeave return HTTP 200 setelah berhasil verifikasi & update. Jika gagal, return non-200 → Midtrans retry.
- **Anti double-charge:** Sebelum checkout, frontend cek pending order via `GET /api/account/pending-order` — jika ada order `checkout_created` aktif, customer harus resume/cancel dulu. Setelah webhook settlement, server auto-cancel order `checkout_created` lain milik user yang sama.
- **Order ID format:** `ww_{12charUserIdPrefix}_{10charSha256Hash}` — hash dari planId + billingCycle + timestamp + random bytes. Contoh: `ww_a1b2c3d4e5f6_f8k2lm9p0q`
- **Tidak ada Snap popup:** WebWeave menggunakan Invoice API yang menghasilkan payment link. Customer redirect ke halaman Midtrans, bukan popup di website WebWeave.
- **Data pembayaran:** WebWeave TIDAK menyimpan, memproses, atau mengakses data kartu/debit. 100% diproses Midtrans.
- **Refund:** WebWeave yang menginisiasi refund via dashboard Midtrans jika memenuhi syarat (double charge / downtime >72h). Tidak ada auto-refund.